Task 1 – State Concepts Deep Dive & Visual Mapping Tip Calculator – Jetpack Compose

Introduction

Jetpack Compose is a modern UI toolkit for Android that follows a declarative programming model. In Compose, the UI is built based on state. When the state changes, the UI automatically updates through a process called recomposition.

In this project, I implemented a Tip Calculator application that takes user input (bill amount and tip percentage), allows optional rounding using a switch, and dynamically updates the calculated tip. This project demonstrates how state flows through composables and how user interaction triggers UI updates.

2 Key Concepts in State Management

2.1 State

Definition:
State is a value that represents the current data of the UI. When state changes, Compose automatically redraws the affected UI elements.

In my project, the following are state variables:

- `amountInput`
- `tipInput`
- `roundup`

```kotlin
var amountInput by remember { mutableStateOf("") }
```

Here:

- `amountInput` stores the user-entered bill amount.
- When the value changes, the UI updates automatically.

State makes the UI reactive instead of manually refreshing views.

2.2 remember()

Definition:
`remember()` keeps a value stored across recompositions. Without it, values would reset every time the UI refreshes.

Example:

```kotlin
var roundUp by remember { mutableStateOf(false) }
```

If `remember()` was not used:

- The switch would reset to OFF during recomposition.
- User experience would break.

Thus, remember ensures UI stability.

2.3 mutableStateOf()

Definition:
`mutableStateOf()` creates a mutable state holder. When its value changes, Compose schedules recomposition.

Example:

```kotlin
var tipInput by remember { mutableStateOf("") }
```

When user types in TextField:

- `tipInput` changes
- `calculateTip()` runs again
- Result Text updates

This is automatic and reactive behavior.

2.4 Recomposition

Definition:
Recomposition is the process where Compose redraws parts of the UI when state changes.

In this project:

When:

- Bill amount changes
  OR
- Tip percentage changes
  OR
- Round-up switch toggles

Then:

- `tip` value recalculates
- The `Text()` displaying tip updates

Example:

```kotlin
Text(
    text = stringResource(R.string.tip_amount, tip)
)
```

3 Interaction Scenario

The selected interaction scenario is:

1. User enters bill amount.
2. User enters tip percentage.
3. User optionally toggles “Round Up”.
4. The tip amount is calculated and displayed.
5. UI updates instantly whenever input changes.

4 State Flow Diagram

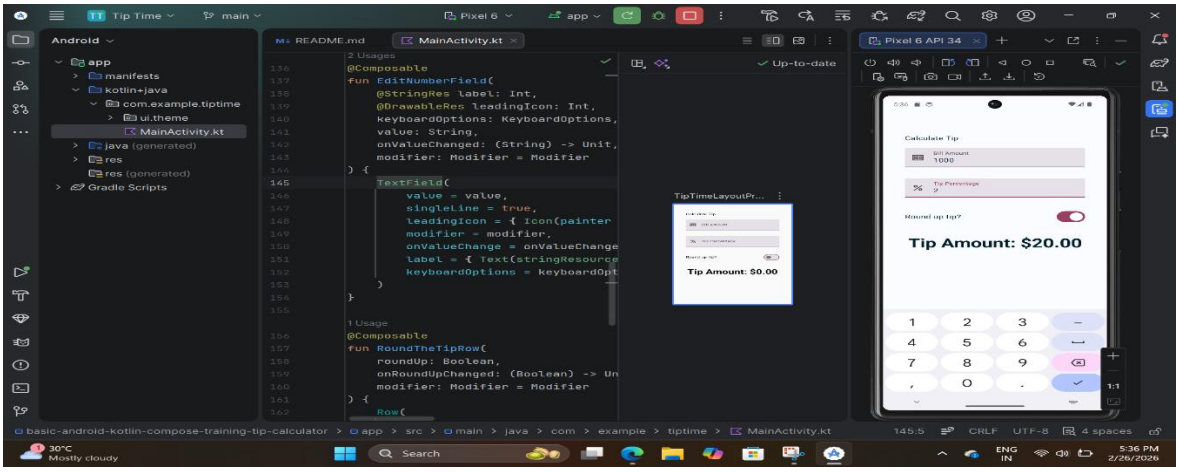
User Input (TextField)
↓
amountInput / tipInput (State Variables)
↓
Conversion to Double
↓
calculateTip()
↓
tip (Formatted String)
↓
Text UI Updates

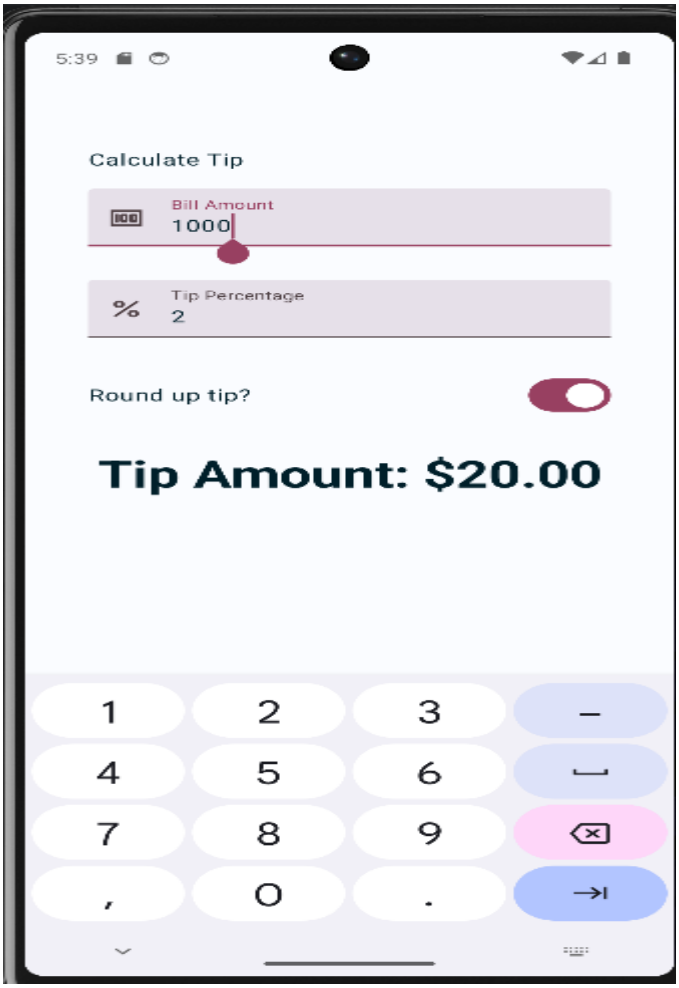
Also show:

Switch Toggle
↓
roundUp (false → true)
↓
calculateTip()
↓
Rounded Tip
↓
UI Updates

This proves you understand state flow visually.

4 Application Output Screenshots





5 State Handling Strategy Table

UI Component	State Variable	Type	Where Defined	Purpose
Bill Amount TextField	amountInput	Mutable State	TipTimeLayout()	Stores bill input
Tip Percentage TextField	tipInput	Mutable State	TipTimeLayout()	Stores tip %
Round Up Switch	roundUp	Hoisted State	TipTimeLayout()	Controls rounding logic
Result Text	tip	Derived State	TipTimeLayout()	Displays calculated tip

6 Reflection

State hoisting is important because it centralizes data management and makes composables reusable and predictable. If state is deeply nested inside child components, the parent cannot properly control UI behavior, leading to inconsistent updates. For example, if the roundUp state were defined inside RoundTheTipRow, the parent composable might not detect changes correctly, causing the tip amount not to update. Mismanaged state can lead to bugs such as UI not updating, incorrect calculations, or reset values during recomposition. Proper state organization improves scalability and reduces future maintenance problems.

Task 2 — Input, Validation & UI Feedback Plan Feature: Expense Splitter with Validation

Feature Scenario

Feature Name: Expense Splitter with Round-Up Option

This feature allows users to enter the total bill amount and the number of people splitting the bill. The user can optionally enable a round-up toggle to round the per-person amount. The app validates inputs before calculation and provides real-time feedback for errors.

This feature is useful because it ensures fair expense sharing and prevents incorrect calculations due to invalid input.

Input & State Variables Listing

Input Fields and State Variables

UI Input Component	State Variable	Data Type	Purpose
Bill Amount TextField	billAmountInput	String	Stores total bill
Number of People TextField	numPeopleInput	String	Stores number of people
Round Up Switch	roundUpSwitch	Boolean	Controls rounding
Error for Bill	billError	String?	Stores validation error
Error for People	peopleError	String?	Stores validation error
Calculate Button Enabled			

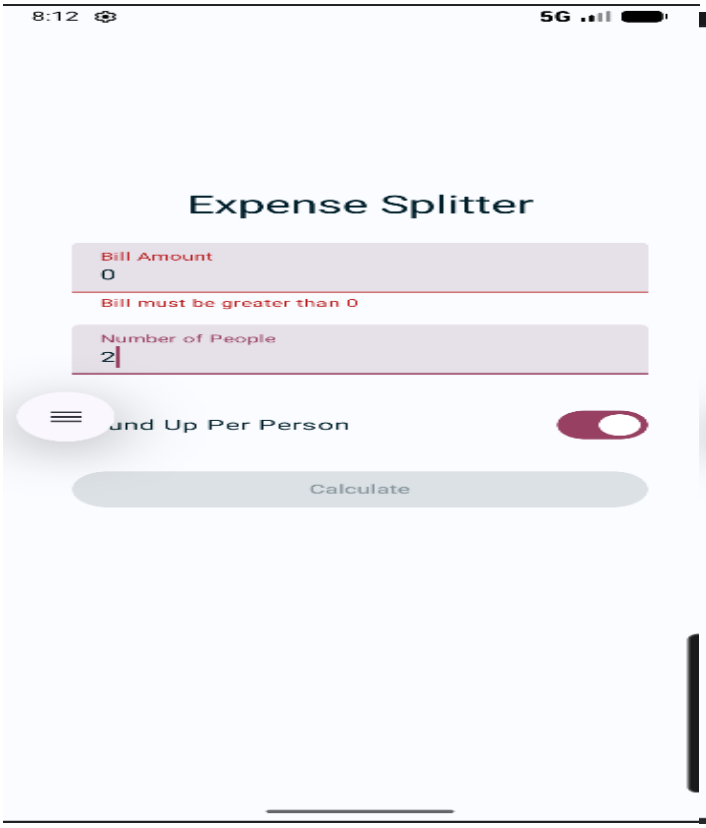
UI Feedback & Feedback States

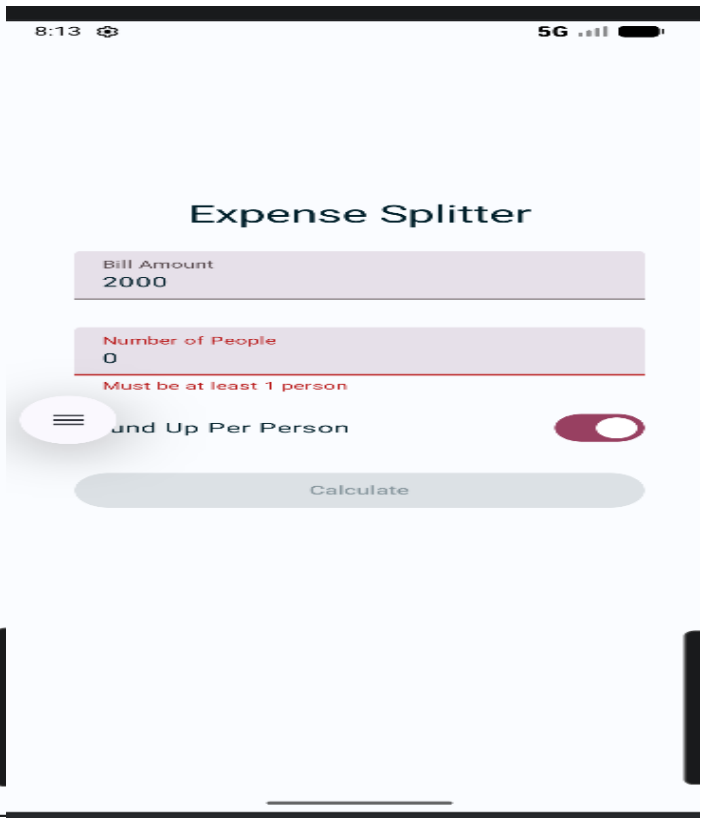
Bill Amount Validation Feedback

Condition	UI Feedback
Empty	Show error: "Bill amount cannot be empty"
≤ 0	Show error: "Bill amount must be greater than 0"
Valid	Remove error and enable calculation

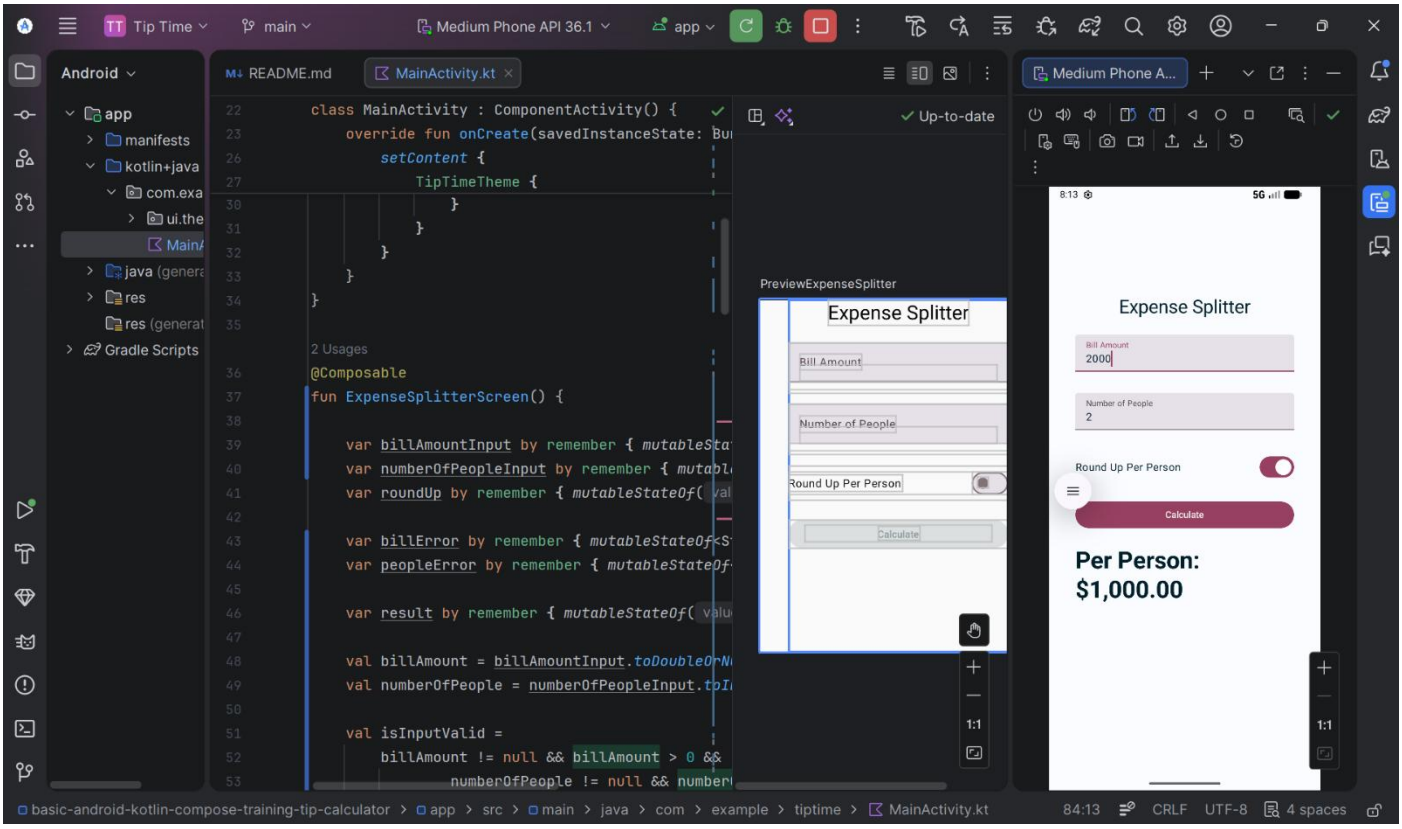
UI Behavior:

- TextField border turns red
- Error text appears below TextField





Wireframe & Flow Sketch



State Planning Overview

State Variable	Type	Triggers Recomposition?	Why Needed
billAmountInput	Mutable State	Yes	Store user input
numPeopleInput	Mutable State	Yes	Store people count
roundUpSwitch	Mutable State	Yes	Control rounding
billError	Mutable State	Yes	Show error message
peopleError	Mutable State	Yes	Show error
result	Derived State	Yes	Display calculation

Reflection

Validation and user feedback are essential because they prevent incorrect data from entering the system and ensure reliable state updates. Without validation, users might enter invalid values such as zero or negative numbers, leading to crashes or incorrect calculations. In a reactive UI like Compose, invalid state can trigger incorrect recompositions and inconsistent UI behavior. Proper validation ensures predictable state transitions and improves user experience by guiding users with clear feedback.

Task 3 — Comprehensive Feature Design & Implementation Strategy

Feature Definition

Summary:
The Bill Split & Tip Tracker allows users to enter the total bill amount, number of people, and tip percentage. Users can choose to round up the total amount and optionally save each calculation in a history list.

Target Users:
Friends, families, or colleagues who want to quickly split restaurant bills accurately.

Key Behaviors:

- Accepts multiple user inputs (bill, people, tip %)
- Includes a round-up toggle
- Calculates per-person share
- Displays validation errors
- Saves previous splits in a history list

2. Implementation Strategy

Main Screen Components:

- TextField → Bill Amount
- TextField → Number of People
- TextField → Tip Percentage
- Switch → Round Up
- Button → Calculate
- Button → Save
- Button → Clear
- Text → Result Display
- LazyColumn → History List

B. Button List & State Changes

Calculate Button

- Validates inputs
- Calculates total with tip
- Updates result state

State Model

State Variable	Type	Mutable	Initial Value	Description
billAmount	String	Yes	""	User input
numPeople	String	Yes	""	User input
tipPercentage	String	Yes	""	User input
roundUp	Boolean	Yes	false	Toggle state
result	Double	Yes	0.0	Calculated value
errorMessage	String	Yes	""	Validation feedback
historyList	MutableList<String>	Yes	emptyList()	Saved splits

Derived State:

- totalWithTip = bill + (bill * tip / 100)
- perPerson = totalWithTip / numPeople

Pseudo-Code Snippet

Kotlin

```kotlin
if (billAmount.isBlank() || numPeople.isBlank() || tipPercentage.isBlank()) {
    errorMessage = "All fields are required"
} else if (billAmount.toDouble() <= 0) {
    errorMessage = "Bill must be greater than 0"
} else if (numPeople.toInt() <= 0) {
    errorMessage = "Number of people must be at least 1"
} else {
    val bill = billAmount.toDouble()
    val people = numPeople.toInt()
    val tip = tipPercentage.toDouble()

    var total = bill + (bill * tip / 100)

    if (roundUp) {
        total = kotlin.math.ceil(total)
    }

    result = total / people
    errorMessage = ""
}
```

Reflection

The most complex part of planning this feature was managing multiple states and ensuring proper validation before calculation. Handling edge cases like blank inputs and zero division required careful logic design.

This task improved my understanding of stateful Compose UI and how derived states work. I gained confidence in structuring validation logic and planning UI feedback.

In the future, I will avoid mixing UI code with business logic and instead separate concerns clearly for better maintainability.